

Mecanum-Ansteuerung im Simulator

Praktikumsversuch 1 — Intelligente Robotik

Betreuung: Robotik-Labor

Version 0.1 — Wintersemester

Sie schreiben die *Ansteuerung der vier Mecanum-Räder* eines mobilen Roboters. Dazu übersetzen Sie eine gewünschte Körpergeschwindigkeit in vier Radgeschwindigkeiten in rad/s und veröffentlichen sie auf ein ROS-Thema. Ein kleiner 2D-Simulator (Pygame, rund 1400 Zeilen Python) fährt Ihren Code in einer von drei Umgebungen und liefert Odometrie, LIDAR und eine globale Position zurück. Der Simulator ist absichtlich winzig: Sie können die Physik in einer halben Stunde lesen und sind damit in der Lage, ihn für eigene Fragen zu verändern.

Inhaltsverzeichnis

1 Versuchsziel und Vorfragen

1.1 Ziel

Nach dem Versuch können Sie

- die Kinematik eines Mecanum-Fahrwerks aus der Rolleranordnung herleiten und die Vorzeichen begründen (statt sie zu kopieren),
- den Unterschied zwischen inverse und forward Kinematik sowie deren pseudo-inverse Auflösung erklären,
- einen ROS-2-Knoten schreiben, der Themen liest (`odom`, `scan`, `gps`) und veröffentlicht (`cmd_vel`, `wheel_speeds`),
- die Drift einer Dead-Reckoning-Odometrie messen und einordnen.

Gearbeitet wird allein pro Person: ein Robotername pro Teilnehmer, ein Knoten pro Roboter. Der Bewerter misst ausschließlich über die Themen — wie Sie implementieren, steht Ihnen frei.

1.2 Vorfragen (vor dem Praktikumsstermin rechnen)

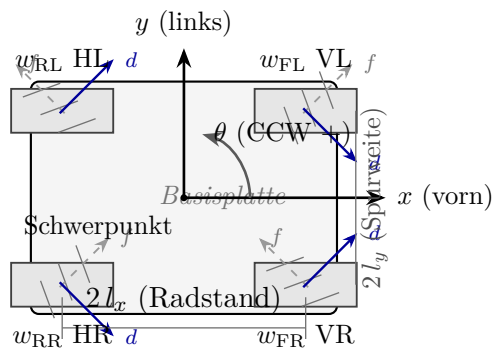
Verwenden Sie die Simulatorwerte aus ??: $l_x = 0,14\text{ m}$, $l_y = 0,13\text{ m}$, $r = 0,05\text{ m}$, $a = l_x + l_y = 0,27\text{ m}$, maximale Radgeschwindigkeit $w_{\max} = 12\text{ rad/s}$. Lösungen: ??.

1. **Vorwärts.** Wie schnell dreht jedes der vier Räder bei $v_x = 0,4 \text{ m/s}$, $v_y = \omega = 0$?
2. **Seitwärts.** Gesucht $v_y = +0,25 \text{ m/s}$ (nach links). Berechnen Sie die vier Raddrehzahlen und erklären Sie, warum zwei Räder negativ sind, obwohl der Roboter nicht dreht.
3. **Drehen auf der Stelle.** Für $\omega = +1,2 \text{ rad/s}$ (gegen den Uhrzeigersinn): welche Vorzeichen haben VL und HL — und warum dieselben?
4. **Diagonalfahrt.** $v_x = v_y = 0,3 \text{ m/s}$. Ein Rad steht still. Welches? Welche Konsequenz hat das für die höchstfahrbare Diagonaleschwindigkeit, wenn alle Räder mit $w_{\max}$ begrenzt sind?
5. **Rückwärtsrechnen.** Die Räder messen $[w_{\text{FL}}, w_{\text{FR}}, w_{\text{RL}}, w_{\text{RR}}] = [8, 0, 0, 8] \text{ rad/s}$. Bestimmen Sie v_x , v_y , ω und geben Sie Fahrtrichtung und Betrag an.

2 Das Mecanum-Prinzip

2.1 Rolleranordnung

Ein Mecanum-Rad ist ein normales Rad, dessen Mantel aus schräg (45°) montierten Rollen besteht. Es kann damit (i) wie ein normales Rad angetrieben in seine Wirkrichtung d rollen und (ii) senkrecht dazu, f , frei gleiten. In der Draufsicht zeigen die vier Wirkrichtungen nach vorn-innen; es entsteht ein X — die *X-Anordnung* (Abbildung ??).



Rand-und-Vorn-Regel: x nach vorn, y nach *links*, θ gegen den Uhrzeigersinn
 Rollachsen-Pfeile d ergeben in der Draufsicht ein X (*X-Anordnung*).
 f = Freigleit-Richtung (dreht das Fahrzeug auf der Stelle, bleibt unvermittelt).
 Radreihenfolge in allen Themen: $[w_{\text{FL}}, w_{\text{FR}}, w_{\text{RL}}, w_{\text{RR}}] = [\text{VL}, \text{VR}, \text{HL}, \text{HR}]$

Abbildung 1: Mecanum-Fahrwerk von oben. Blaue Pfeile d = angetriebene Wirkrichtung, gestrichelte Pfeile f = Freigleit-Richtung, schraffierte Streifen = Rollen. Achsen und Vorzeichen sind exakt die des Simulators.

2.2 Geschwindigkeitszerlegung an einem Rad

Der Schwerpunkt habe die Körpergeschwindigkeit $\xi = (v_x, v_y, \omega)^\top$; das Rad i sitze an der Stelle $\mathbf{p}_i = (\pm l_x, \pm l_y)$. Sein Kontaktpunkt bewegt sich mit

$$\mathbf{v}_i = \begin{pmatrix} v_x \\ v_y \end{pmatrix} + \omega \begin{pmatrix} -p_{i,y} \\ p_{i,x} \end{pmatrix}, \quad \text{z. B. für VL: } \mathbf{v}_{\text{FL}} = \begin{pmatrix} v_x - l_y \omega \\ v_y + l_x \omega \end{pmatrix}. \quad (1)$$

Das Rad liefert nur in Wirkrichtung Drehzahl, in Freigleit-Richtung überträgt es nichts. Mit den Wirkrichtungen aus ??, $d_{\text{FL}} = (1, -1)$, $d_{\text{FR}} = (1, 1)$, $d_{\text{RL}} = (1, 1)$, $d_{\text{RR}} = (1, -1)$ (Projektion ohne Normierung — der Faktor $\sqrt{2}$ steckt in der effektiven Radradius-Kalibrierung), folgt $r w_i = \mathbf{v}_i \cdot d_i$:

$$r w_{\text{FL}} = (v_x - l_y \omega) - (v_y + l_x \omega) = v_x - v_y - a \omega \quad (2)$$

$$r w_{\text{FR}} = (v_x + l_y \omega) + (v_y - l_x \omega) = v_x + v_y + a \omega \quad (3)$$

$$r w_{\text{RL}} = (v_x - l_y \omega) + (v_y + l_x \omega) = v_x + v_y - a \omega \quad (4)$$

$$r w_{\text{RR}} = (v_x + l_y \omega) - (v_y - l_x \omega) = v_x - v_y + a \omega \quad (5)$$

mit dem Gier-Hebelarm $a = l_x + l_y$. Das ist die **inverse Kinematik** — Ihre Aufgabe in T1. Als Matrix (Radreihenfolge [FL, FR, RL, RR]):

$$\mathbf{w} = J \xi, \quad J = \frac{1}{r} \begin{pmatrix} 1 & -1 & -a \\ 1 & 1 & a \\ 1 & 1 & -a \\ 1 & -1 & a \end{pmatrix}. \quad (6)$$

2.3 Vorwärtskinematik als Pseudo-Inverse

Gesucht ist ξ aus gemessenen Raddrehzahlen, also $\mathbf{w} = J\xi$ mit vier Gleichungen und drei Unbekannten. Da J Spaltenrank 3 hat, ist die Moore-Penrose-Lösung ⁺die Pseudo-Inverse $J^+ = (J^\top J)^{-1} J^\top$. Es ist $J^\top J = \frac{1}{r^2} \text{diag}(4, 4, 4a^2)$ — die drei Freiheitsgrade sind also *entkoppelt*, und es ergibt sich

$$J^+ = \frac{r}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{a} & \frac{1}{a} & -\frac{1}{a} & \frac{1}{a} \end{pmatrix} \Rightarrow \begin{aligned} v_x &= \frac{r}{4} (w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} + w_{\text{RR}}) \\ v_y &= \frac{r}{4} (-w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} - w_{\text{RR}}) \\ \omega &= \frac{r}{4a} (-w_{\text{FL}} + w_{\text{FR}} - w_{\text{RL}} + w_{\text{RR}}) \end{aligned} \quad (7)$$

Der Simulator benutzt ?? für die Odometrie: Was die Räder tatsächlich machen, entscheidet, wo der Roboter ankommt — nicht das, was Sie wollten. Deshalb wirkt sich ein falsches Vorzeichen in T1 sofort als Fehlfahrt aus.

Vorzeichen-Box (Merksatz).

x nach vorn, y nach **links**, θ gegen den Uhrzeigersinn positiv. Radreihenfolge in *allen* Themen: [FL, FR, RL, RR] = [VL, VR, HL, HR]. Drehzahl in rad/s, Körpergeschwindigkeit in m/s bzw. rad/s.

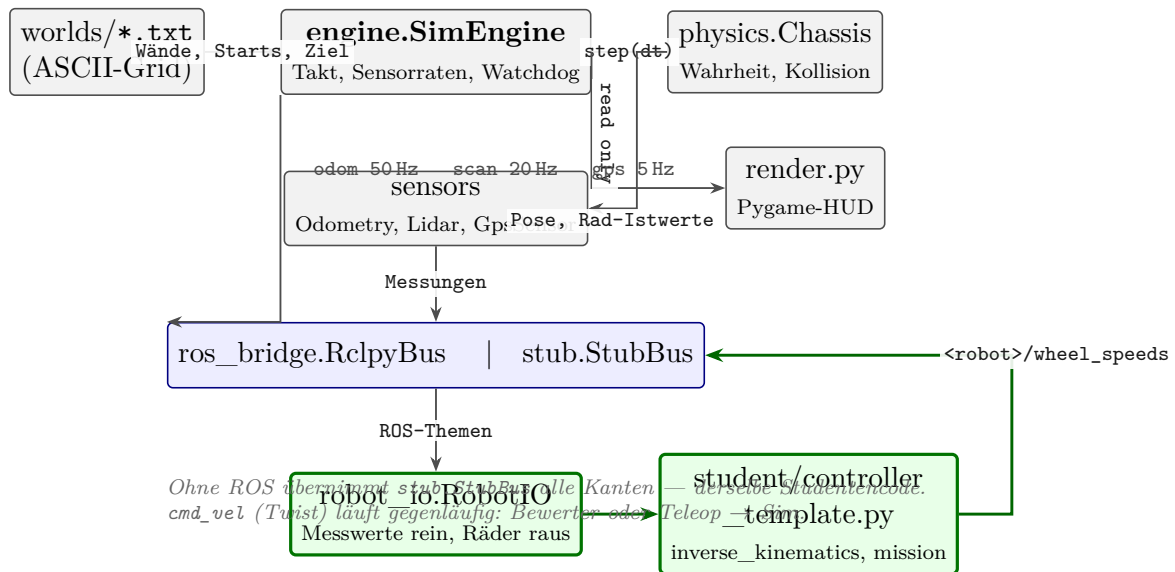


Abbildung 2: Datenfluss. Der geschlossene Regelkreis läuft über `wheel_speeds`: Nur Ihre vier Radzahlen bestimmen die Pose. Der Renderer ist rein lesend.

3 Der Simulator

3.1 Architektur

Der Simulator existiert in zwei Spielarten, am selben Code: *mit* ROS 2 (`ros_bridge.py` → echter DDS-Bus) und *ohne* ROS (`stub.py` → Bus im selben Prozess). Für Sie ist der Studentencode identisch.

3.2 Themen und Typen

Frames: `map` (Welt), `odom` (Odometrie-Ursprung), `base_link` (Roboter), `laser`. Roboternamen tragen ihre Themen als Namensraum.

3.3 Was verrauscht, was exakt ist

Exakt (deterministisch, kein Rauschen) sind nur die *Wahrheit* in `physics.py` und — falls eingeschaltet — das Thema `<robot>/truth`. Schlupf wird nicht modelliert: vier Radzahlen ergeben eindeutig (v_x, v_y, ω) nach ??.

3.4 pass-through-Modus und wheels-Modus

Damit der erste Tag nicht an der Kinematik scheitert, startet jeder Roboter im **pass-through**-Modus: Kommt `cmd_vel` und *nie* eine `wheel_speeds`-Meldung, rechnet die Simulation die Twist selbst in vier Räder — der Roboter fährt, als wäre Ihre Lösung schon richtig. Senden Sie die erste `wheel_speeds`-Meldung, kippt der Roboter für immer in den **wheels**-Modus: jetzt gilt ausschließlich, was Sie rechnen (Modus steht in `/sim/robots` und im HUD). Ein kaputtes Vorzeichen ist damit sofort sichtbar — und genau so gemeint.

Tabelle 1: ROS-Schnittstelle (Pfad <robot> = Robotername).

Thema	Typ (ROS 2)	Bedeutung
<robot>/cmd_vel	geometry_msgs/msg/Twist	Soll-Körpergeschwindigkeit; $v_y > 0$ heißt <i>links</i>
<robot>/wheel_speeds	std_msgs/msg/Float64MultiArray (4)	vier Radzahlen in rad/s — der Übungsteil
<robot>/odom	nav_msgs/msg/Odometry	Dead Reckoning aus <i>gemessenen</i> Raddrehzahlen, verrauscht
<robot>/scan	sensor_msgs/msg/LaserScan	360 Strahlen über 2π ; Strahl 0 zeigt in Fahrtrichtung, wachsender Index gegen den Uhrzeigersinn; kein Treffer = range_max
<robot>/gps	geometry_msgs/msg/PoseStamped	globale Position (UWB/MoCap-Charakter), verrauscht
<robot>/truth	geometry_msgs/msg/PoseStamped	exakte Pose — nur, wenn debug_truth eingeschaltet ist
<robot>/mission_state	std_msgs/msg/String	idle , running , done , failed :<grund>
/sim/robots	std_msgs/msg/String (JSON)	alle Roboter mit Farbe, Marker, Motorvariante, Modus
/sim/world	std_msgs/msg/String (JSON)	Weltname, Größe, Ziel, Startposes % TODO(integrator): in CONTRACT 3 ergänzen
/sim/task	std_msgs/msg/String	aktiver Auftrag des Bewerter
/sim/spawn_robot	Service, siehe ??	Roboter hinzufügen (eindeutiger Name Pflicht)
/sim/despawn_robot	Service, siehe ??	Roboter entfernen
/sim/reset	std_srvs/srv/Trigger	alle Roboter zurück an den Start
/clock	roscpp_msgs/msg/Clock	Simulationszeit (use_sim_time)

4 Inbetriebnahme

4.1 Installation

```

sudo apt install python3-pygame          # Ubuntu, reicht fuer den Stub-
    Modus
git clone <repo>                          # folgt im naechsten Schritt des
    Praktikums
cd mecanum-lab
./lab run --world maze --robot alice      # Sim + Ihr Knoten, EIN Prozess,
    kein ROS

```

Mit ROS 2 (getestet mit Kilted/Jazzy, Ubuntu 24.04):

```
source /opt/ros/$ROS_DISTRO/setup.bash
```

Tabelle 2: Parameter aus `config/default.json` (Auszug).

Größe	Wert	Bemerkung
Physikrate	50 Hz	festе Schrittweite, damit Läufe reproduzierbar sind
Odometrie	50 Hz	Rauschen $\sigma_{\text{wheel}} = 0,04 \text{ rad/s}$ auf jeder Radmessung $\Rightarrow$ integrierte Drift; $\sigma_{xy} = 1,5 \text{ mm}$, $\sigma_{\theta} = 1,2 \text{ mrad}$ auf dem Ausgang
LIDAR	20 Hz	360 Strahlen, $0 \dots 8 \text{ m}$, $\sigma = 15 \text{ mm}$; Roboter sehen sich in Versuch 1 nicht
Globale Position	5 Hz	$\sigma_{xy} = 6 \text{ cm}$, $\sigma_{\theta} = 0,03 \text{ rad}$
Motor	$\tau = 0,06 \text{ s}$	Trägheit 1. Ordnung, $ \dot{w} \leq 40 \text{ rad/s}^2$, $ w \leq 12 \text{ rad/s}$
Kommando-Wachdog	0,35 s	ohne neue Meldung rollen die Räder aus
Kollision	Kreis $r = 0,21 \text{ m}$	Position wird aus der Wand geschoben, Berührungen zählen (<code>contacts</code>)

```
./lab sim --world production --robots alice      # Sim solo, veröffentlicht
ROS-Themen
```

4.2 Ausprobieren

```
./lab teleop --robot alice      # Twist-Fernsteuerung auf cmd_vel
./lab robots                    # wer fahrt, in welchem Modus?
ros2 topic list
ros2 topic echo /alice/odom --once
ros2 topic hz /alice/scan      # ~20 Hz
ros2 topic echo /alice/wheel_speeds # Ihre vier Radzahlen
```

4.3 Roboter per Service anlegen

Ein Roboter pro Teilnehmer, Name eindeutig (`[a-z][a-z0-9_]{1,23}`). **Achtung ROS 2:** anders als in ROS 1 gibt es *kein* gebautes `std_srvs/SetString`. Der Simulator bietet deshalb zwei Wege, die Aufrufer gleichermaßen funktionieren:

1. **Eigenes Interface** (empfohlen, eine einmalige Übung in `.srv`-Definitionen):

```
# interfaces/mecanum_lab_interfaces/srv/SpawnRobot.srv
string name          # eindeutig
string variant       # "" = automatisch (stockslowfastagile)--bool
    successstring messageuint8 indexstring colorstring markerfloat64
    xfloat64 yfloat64 theta

colcon build --packages-select mecanum_lab_interfaces
source install/setup.bash
ros2 service call /sim/spawn_robot mecanum_lab_interfaces/srv/SpawnRobot \
    "{name: alice, variant: ''}"
```

2. **JSON-Fallback ohne Interface-Bau:** Handshake über Themen, `/sim/spawn_robot/request` (`std_msgs/String`, JSON) → Antwort auf `/sim/spawn_robot/result`, Timeout 2s.

Beide Wege kapseln `./lab spawn -name alice` und `./lab despawn -name alice`; der Bewerter benutzt ebenfalls diesen Weg.

4.4 Stolperfallen

- **Kein DISPLAY** (SSH, CI): `SDL_VIDEODRIVER=dummy` oder `./lab sim -headless`.
- **ROS nicht gerodet:** `ros2` fehlt → alles läuft automatisch über den Stub-Bus; `ros2 topic list` zeigt dann natürlich nichts.
- **use_sim_time:** Knoten mit Wanduhrzeit statt `/clock` vergleichen Zeitstempel falsch → `-ros-args -p use_sim_time:=true`.
- **Name bereits vergeben:** `success=false`, `message="name vergeben"` — erst `./lab despawn`, dann neu.
- **Namensraum:** Themen hängen am Roboternamen, `/alice/odom` `/bob/odom`. Wer keinen Roboternamen übergibt, wartet ins Leere.
- **Funkstille:** ohne Meldung innerhalb 0,35s rollen die Räder aus — Ihr Knoten muss mit ≥ 10 Hz senden.

5 Aufgaben

Alle Schwellen stehen in `config/tasks.json` und sind dort änderbar; der Bewerter misst nur über Themen. Ausgeführt wird in der Produktionsumgebung, `./lab grade -robot alice -task alle` (Punkte pro Auftrag in ??).

T1 — Inverse Kinematik (30 Punkte)

Sie ergänzen in `student/controller_template.py` genau eine Funktion. Der Rest des Knotens (Themen, Takt, Wachdog) ist vorbereitet:

```
def inverse_kinematics(vx, vy, omega):
    """[VL, VR, HL, HR] in rad/s Konvention aus Abschnitt 2."""
    a = LX + LY # TODO: Hebelarm fuer Gier
    return [(vx - vy - a * omega) / R, # TODO: VL
            (vx + vy + a * omega) / R, # TODO: VR
            (vx + vy - a * omega) / R, # TODO: HL
            (vx - vy + a * omega) / R] # TODO: RR
```

Teilschritte. (1) Stub-Modus starten, Teleop benutzen, im HUD die Räder beobachten. (2) Ein Vorzeichen nach dem anderen abprüfen: nur v_x , dann nur v_y , dann nur ω . (3) `./lab grade -task kinematik` laufen lassen. **Lernziel:** Vorzeichen aus der Rollergeometrie verstehen, nicht raten.

Der Simulator selbst rechnet (Auszug aus `mecanum_lab/physics.py`, Ihre Rechnung muss auf dieselben Werte kommen):

Tabelle 3: Aufträge, Punkte, Erfolgsbedingungen.

ID	Inhalt	Grenze	Punkte
kinematikT1:	cmd_vel → vier Räder. Drei Phasen zu je 3 s: $v_x = 0,3$; $v_y = 0,3$; $\omega = 0,6$.	$\Delta x > 0,35$ m $\Delta y > 0,35$ m $\Delta \theta > 0,9$ rad je $ \text{Rest} < 0,12$ m	30
quadrat	T2: Quadratfahrt, vier Seiten 1,0 m, 90° CCW, nur mit Odometrie.	Abschluss $< 0,20$ m, $\Delta \theta < 15^\circ$, Weg 3,2 ... 9,0 m, 0 Berührungen, < 90 s	30
korridor	T3: Ziel der Welt anfahren, LIDAR verhindert jede Wandberührung.	Zielfehler $< 0,25$ m, 0 Berührungen, seitlicher Abstand $> 0,16$ m bei Geradaus ($ \omega < 0,25$)	30
gps_anfahrt	T4 (Bonus): mit GPS-Rückführung zur eigenen Startpose.	Fehler $< 0,25$ m, 0 Berührungen, < 90 s	10

```
def inverse_kinematics(g, vx, vy, omega) -> list:
    """Kuerpergeschwindigkeit -> vier Solldrehzahlen [VL, VR, HL, HR] in rad /s."""
    a = g.arm
    return [(vx - vy - a * omega) / g.r, (vx + vy + a * omega) / g.r,
            (vx + vy - a * omega) / g.r, (vx - vy + a * omega) / g.r]

def forward_kinematics(g, wheels) -> tuple:
    """Vier Radgeschwindigkeiten -> (vx, vy, omega); exakte Umkehrung der obigen Zeilen."""
    fl, fr, rl, rr = wheels
    a = g.arm
    return (g.r / 4 * (fl + fr + rl + rr),
            g.r / 4 * (-fl + fr + rl - rr),
            g.r / (4 * a) * (-fl + fr - rl + rr))
```

T2 — Quadratfahrt mit Odometrie (30 Punkte)

Erweitern Sie `mission(rob, task)`: vier Seiten 1,0 m, jeweils 90° gegen den Uhrzeigersinn, Rückführung über `rob.odom()`. Melden Sie `done`, wenn Sie wieder an der Startpose sind. Typischer Weg: Geradaus-P-Drehregelung auf den Achsenfehler, Totband für

die Drehung, Zwischenziele in der Odometrie gespeichert. Messen Sie zusätzlich Ihren Abschlussfehler und kommentieren Sie, woher er kommt (Drift, Trägheit, Winkel darstellung).

T3 — Ziel anfahren, der LIDAR schützt (30 Punkte)

Ziel: goal der Welt (steckt als JSON in `/sim/world`). Richtung und Strecke aus der Odometrie, Sicherheit aus `rob.scan()`: seitlicher Abstand $> 0,16$ m, *keine* Wandberührung (`contacts=0`). Sinnvolle Bausteine: Vokablen aus Strahlmittelwerten (nicht einzelne Strahlen!), Geschwindigkeitsreduktion mit dem vorderen Mindestabstand, Not-Stopp.

T4 — Bonus: GPS statt Odometrie (10 Punkte)

Rückweg zur eigenen Startpose, aber geregelt auf `rob.gps()`. Beobachten und kommentieren Sie: Totzone eines P-Reglers, Messrauschen als Zittern, 5 Hz gegen 50 Hz Odometrie.

6 Abgabe und Bewertung

- Abgegeben wird `student/<name>_controller.py` in Ihrem Branch/Ordner; Robotername = Dateiname = Teilnehmerkürzel.
- Der Autograder ist maßgeblich: `./lab grade -robot <name> -task alle -json ergebnis.json`. Ein Lauf pro Abgabe, Werte aus ??.
- Dazu eine kurze README (≤ 20 Zeilen): Vorgehen, bekannte Probleme, was Sie anders machen würden.
- Selbsttest vor der Abgabe: `./lab run -robot test -controller student/<name>_controller. -headless und python3 -m pytest tests -q`.

Mussertext für die Commit-Meldung

T2: Quadratfahrt mit Odometrie (1.0 m, 90 Grad CCW)

- `inverse_kinematics` auf Konvention Abschnitt 2 korrigiert (vy vorzeichenrichtig)
- Drehrueckfuehrung ueber `odom.theta`, Totband 0.03 rad, P-Gewinn 1.6
- bekannt: Abschlussfehler ~ 0.15 m durch Drift; GPS (T4) waere die naechste Stufe

Bewertungsraster

Hilfsmittel und Selbstständigkeit

Abschreiben von Lösungsteilen (auch aus generierten Antworten anderer Personen oder Sprachmodelle) führt zum Rücktritt des Versuchs; diskutieren dürfen Sie alles. Hilfsmittel: Skript, diese Anleitung, `docs/`, Ihre eigenen Notizen. Die Musterlösung

Kriterium	Punkte	geprüft durch
T1 Kinematik (drei Phasen je 10 P)	30	Autograder
T2 Quadratfahrt (Abschluss 20, Fahrqualität 10)	30	Autograder
T3 Ziel ohne Berührung (Zielfehler 20, Abstand 10)	30	Autograder
T4 Bonus GPS	10	Autograder
kurzes Resümee im README	0–5	Betreuung

`student/solution.py` ist im Repo und wird *erst nach Abgabeschluss* freigegeben %
 TODO(integrator): Freigabezeitpunkt festlegen. Die vier Aufträge sind hintereinander
 zu bearbeiten; T1 ist Voraussetzung für alles andere.

7 Ausblick

- **Versuch 2 — Odometrie-Fusion:** Odometrie (50 Hz, Drift) mit GPS (5 Hz, verrauscht) zu einem Schätzer kombinieren; Kompass fehlt bewusst.
- **Versuch 3 — Pfadfolgen:** Rundkurs-Welt, Follow-the-Corridor mit LIDAR, Geschwindigkeitsbegrenzung durch Kurvenradius.
- **Versuch 4 — Mehrroboter:** mehrere Roboter, eigene Namensräume, Kollisionen zwischen Robotern (dafür muss `sensors.py` erweitert werden — genau die Art Änderung, die der kleine Simulator erlaubt).

A Themen-, Typen- und API-Übersicht

Nachrichtenaufbau (Auszug)

```
# /alice/wheel_speeds : std_msgs/msg/Float64MultiArray
msg.data = [w_FL, w_FR, w_RL, w_RR]          # rad/s, Reihenfolge ist Vertrag

# /alice/odom : nav_msgs/msg/Odometry
odom.pose.pose.position.{x,y}                # Weltkarte, Framework odom
odom.pose.pose.orientation.z, .w             # Quaternion um z (theta)
odom.twist.twist.linear.{x,y}                # Koerperam, y nach links
odom.twist.twist.angular.z                   # rad/s, CCW positiv

# /alice/scan : sensor_msgs/msg/LaserScan
scan.angle_min = 0.0                         # Strahl 0 = Fahrtrichtung
scan.angle_increment = 2*pi/360              # wachsender Index: CCW
scan.ranges[i]                               # m; kein Treffer = range_max (8.0)

# /alice/gps : geometry_msgs/msg/PoseStamped (Framework map)
# /sim/world : std_msgs/msg/String, JSON: name, size, goal, spawns, walls
```

Studentische API (robot_io.py)

```
rob = RobotIO("alice")           # ./lab run --robot alice
rob.cmd_vel()                   # Twist oder None
rob.odom(); rob.gps(); rob.scan()
rob.age("odom")                 # Alter der letzten Messung in s
rob.send_wheels([w1, w2, w3, w4])
rob.publish_cmd_vel(vx, vy, omega)
rob.set_mission_state("done")
rob.task(); rob.world(); rob.robots()
serve(module)                   # Runner hinter der einen Zeile im Template
```

Vertrag mit serve: Ist /sim/task leer oder kinematik, wird jede cmd_vel mit inverse_kinematics in Räder übersetzt. Bei einem anderen Auftrag wird genau einmal mission(rob, task) aufgerufen und der Zustand gemeldet. % TODO(integrator): finalen Wortlaut des Templates prüfen

B Formelsammlung

$$\text{IK: } r \begin{pmatrix} w_{\text{FL}} \\ w_{\text{FR}} \\ w_{\text{RL}} \\ w_{\text{RR}} \end{pmatrix} \quad (8)$$

$$\text{FK: } v_x = \frac{r}{4} \sum_i w_i, \quad v_y = \frac{r}{4} (-w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} - w_{\text{RR}}), \quad \omega = \frac{r}{4a} (-w_{\text{FL}} + w_{\text{FR}} - w_{\text{RL}} + w_{\text{RR}}) \quad (9)$$

$$\text{Grenze: } v_{\text{max}} = r w_{\text{max}} = 0,6 \text{ m/s}, \quad \omega_{\text{max}} = \frac{r w_{\text{max}}}{a} = 2,22 \text{ rad/s} \quad (\text{jeweils für } v_y \text{ bzw. reine Drehung}) \quad (10)$$

$$\text{Winkel normieren: } \theta \in (-\pi, \pi], \quad \Delta\theta = \text{wrap}(\theta_{\text{Ziel}} - \theta_{\text{is}}) \quad (11)$$

C Lösungen der Vorfragen

1. Alle vier gleich: $w = v_x/r = 0,4/0,05 = 8 \text{ rad/s}$. a spielt keine Rolle, $\omega = 0$.
2. $w_{\text{FL}} = -v_y/r = -5$, $w_{\text{FR}} = +5$, $w_{\text{RL}} = +5$, $w_{\text{RR}} = -5 \text{ rad/s}$. Die Räder müssen *gegeneinander* arbeiten, weil jede Rolle nur längs d Kraft überträgt: die linke Radseite wird nach hinten, die rechte nach vorn geschoben, die Freigleit- richtungen lassen die Längsbewegung des Fahrzeugs zu. Kein Giermoment, weil sich die Hebel aufheben $(-, +, +, -$ mit Vorzeichen aus ????????).
3. $w = \mp a\omega/r = \mp 0,27 \cdot 1,2/0,05 = \mp 6,48 \text{ rad/s}$, konkret $[-6,48, +6,48, -6,48, +6,48]$. VL und HL haben dasselbe Vorzeichen, weil bei reiner Drehung das Vorzeichen nur von x_i (Radpaar vorne/hinten) und dem Drehsinn abhängt — nicht von links/rechts; die Seitwärtsanteile sind hier Null.

4. $w_{\text{FL}} = w_{\text{RR}} = (0,3 - 0,3)/0,05 = 0$, $w_{\text{FR}} = w_{\text{RL}} = 12 \text{ rad/s}$. Stillstand bei VL/RR ist korrekt. Bei $w_{\text{max}} = 12$ ist die Diagonale gerade grenzwertig: $(v_x - v_y) = 0$, $(v_x + v_y) = 0,6 = r w_{\text{max}}$ — diagonal ist bei $v_x = v_y = 0,3 \text{ m/s}$ Schluss, obwohl längs 0,6 ginge.
5. $v_x = \frac{0,05}{4}(8 + 0 + 0 + 8) = 0,2 \text{ m/s}$, $v_y = \frac{0,05}{4}(-8 + 0 + 0 - 8) = -0,2 \text{ m/s}$ (nach rechts!), $\omega = \frac{0,05}{4 \cdot 0,27}(-8 + 0 - 0 + 8) = 0$. Betrag $\sqrt{2} \cdot 0,2 = 0,283 \text{ m/s}$, Richtung 45° vorn-rechts. Als Soll-Kommando wäre das $(v_x, v_y) = (0,2, -0,2)$.